

予選 解説

2018年12月10日・情報オリンピック日本委員会

第18回日本情報オリンピック 予選

問題 1 ソーシャルゲーム (Social Game)

配点 100点 時間制限 2 sec メモリ制限 256 MB

解説

この問題は、肩慣らしと JOI 予選の出題形式に慣れてもらうことを意図して出題されている。

月曜日から連続でログインしていき、何日目初めて C 枚以上のコインを得られるのかを求めればよい。
これはループを書くことで求められるが、以下のように割り算でも求められる。

小課題 1

C / A の小数点以下を切り上げた回数のログインが必要である。これは $\lceil (C + A - 1) / A \rceil$ に等しい。
($\lceil X \rceil$ は X 以下の最大の整数)

小課題 2

はじめの $\lceil C / (7A + B) \rceil$ 週間は、月曜日から日曜日まで毎日ログインすることになる。

残りの必要なコインの枚数を D とすると、これを日々のログインから得るには $\lceil (D + A - 1) / A \rceil$ 日必要である。この値が 7 より大きくなる場合は、実際は 7 日ログインすることで更に B 枚のコインが得られるため、 7 日で D 枚のコインが得られる。

原文：<https://www2.ioi-jp.org/joi/2018/2019-yo/2019-yo-t1-review.html>

問題 2 すごろくと駒 (Sugoroku and Pieces)

配点 100点 時間制限 2 sec メモリ制限 256 MB

解説

駒の座標を配列で管理し、動きを 1 回ずつシミュレートをする。移動元がゴールマスでないかは $O(1)$ 時間で判定できる。移動先の座標に駒があるかどうかは、残りの駒の座標をすべて見ることで、 $O(N)$ 時間で判定できる。よってこの問題は $O(NM)$ 時間で解ける。

この問題ではこれ以上の高速化は要求されていないが、駒の座標が常に昇順であることを利用すると、移動先の座標に駒があるかどうかは $O(1)$ 時間で判定でき、全体で $O(N+M)$ 時間でこの問題は解ける。

原文：<https://www2.ioi-jp.org/joi/2018/2019-yo/2019-yo-t2-review.html>

問題 3 マルバツスタンプ (Circle Cross Stamps)

配点 100点 時間制限 2 sec メモリ制限 256 MB

解説

マルが 2 個連続、あるいはバツが 2 個連続している箇所は、その 2 個をまたいでスタンプを使うことができない。よって、与えられた文字列 S は、同じ文字が連続する箇所で分割して、分割されたそれぞれの文字列について問題を解けば良い。最終的な答えは、分割されたそれぞれの文字列の解の総和になる。

分割された文字列は、同じ文字が連続することがないので、マルとバツが交互に並んだ文字列になる。このような文字列の解は、文字列長を 2 で割った値を小数点以下切り捨てた値と一致する。これは左から順にマルバツスタンプを使っていくことを考えると容易にわかる。

例えば、"00X0X0X0X0000X0XX" という文字列は

"0", "0X0X0X0X0", "0", "0", "0X0X", "X"

というふうに分割する。

それぞれの文字列について、文字列長を 2 で割った値を小数点以下切り捨てた値は

0, 4, 0, 0, 2, 0

となるので、総和の 6 が答えになる。

この操作を観察すると、文字列を実際に分割しなくても、先頭から順に、マルバツスタンプが使えるならば使う、使えないならば 1 文字のスタンプを使う、というふうに貪欲にスタンプを割り当てても解けることがわかる。この貪欲法を使えば $O(Sの文字数)$ で解を求めることができる。

原文 : <https://www2.ioi-jp.org/joi/2018/2019-yo/2019-yo-t3-review.html>

問題 4 日本沈没 (Japan Sinks)

配点 100点

時間制限 2 sec

メモリ制限 256 MB

解説

小課題 1

海面の高さが整数の時点のみ調べればよい。

海面の高さが t の時、島の数 $A_i > t$ となる i の連続した区間の個数である。これは $O(N)$ 時間で求めることができる。例えば、 $A_0 = 0$ において、 $A_{i-1} \leq t$ かつ $A_i > t$ となる i の個数を数えればよい。

よって $O(N \max A_i)$ 時間で問題を解くことができる。

小課題 2

小課題 1 の解放において、海面の高さが 0 の時点およびある区画の高さと等しい時点のみ調べればよい。よって $O(N^2)$ 時間で問題を解くことができる。

小課題 3

ある状態から、陸地が一つ沈没したとき、それによって島の数が増えるか、減るか、変わらないかはその左右の区画を見ればわかる。どちらも陸地であれば島数は増え、どちらも沈んでいけば島数は減り、そのどちらでもなければ島数は変わらない。

$0, A_1, \dots, A_N$ に現れる数を昇順にソートしたものを $B_0, \dots, B_M$ と置く。

海面の高さが B_{i-1} の時点と B_i の時点では、高さが B_i の区画が沈没すること以外は状態は変わらない。そのような区画それぞれについて上記の場合分けを見れば、海面の高さが B_{i-1} から B_i になる際の島数の増減がわかる。そのような区画の番号たちは、 A をソートする際に番号も一緒にペアにしてソートしておけばそこから取り出せる。

よって、海面の高さが $B_0, \dots, B_M$ の時点での島数を順番に求めていくことができる。各区画について高々一度しか上記の場合分けを見ないので、これは $O(N)$ である。ソートに $O(N \log N)$ かかるので、全体で $O(N \log N)$ 時間でこの問題を解くことができる。

原文：<https://www2.ioi-jp.org/joi/2018/2019-yo/2019-yo-t4-review.html>

問題 5 イルミネーション (Illumination)

配点 100点

時間制限 2 sec

メモリ制限 256 MB

解説

まず、それぞれの木にイルミネーションを飾り付けるかどうかをすべて試すことを考える。このような方法は 2^N 通りあり、飾り付け方が条件を満たすかどうかの判定は単純には $O(NM)$ にかかるため、このアルゴリズムの計算量は $O(2^N \times NM)$ となる。この方法では、10 点を獲得することができる。

より高速な解法として、動的計画法に基づくアルゴリズムがある。たとえば、木 $1, 2, \dots, N$ の順に、イルミネーションを飾り付けるかどうかを決めていくことを考える。このとき、「木 $L_j, \dots, R_j$ のうち 2 つ以上にイルミネーションを飾り付けてはならない」という条件は、「木 R_j について飾り付けるかどうか決めたとき、イルミネーションを飾り付ける木であって、2 番目に最新のものは、木 L_j より前である」と言い換えられる。

これは、動的計画法の状態として、イルミネーションを飾り付ける木であって、最新のもののおよび、2 番目に最新のものの組み合わせをとり、各 j に対して、 R_j についての更新後に「2 番目に最新のものは L_j より前」という条件を覚えておくことができる。この方法の計算量は $O(N^3 + M)$ となり、40 点を獲得できる。

木 i を飾り付けたとき、その次に飾り付けることができる木は、ある X_i に対して「番号が X_i 以上の木」として表すことができる。明らかに、任意の i に対して、 X_i は $i+1$ 以上である。各 j に対し、木 $L_j, \dots, R_j$ に対しては X の値は $R_j + 1$ 以上である。逆に、すべての j に対してこの更新を行った後には、実際にこの値以上の番号を持つ木を飾り付けることはできることがわかる。よって、この値を $O(NM)$ かけて前計算した後、「最後に飾り付けた木の番号」を状態に持つ動的計画法を行うと、 $O(NM)$ でこの問題を解くことができ、70 点を獲得できる。

Segment Tree などのデータ構造を用いると、前計算や動的計画法の更新を高速化することができ、 $O((N+M) \log N)$ などで問題を解くことはできる。ここでは、より簡潔に $O(N+M)$ の計算量で解く解法を紹介する。前のアルゴリズムにおいて、前計算で各 j に対し L_j についてのみ X の値を $R_j + 1$ との最大値をとって更新しておき、最後に $i = 1, 2, \dots, N$ の順に、 X_i を X_{i-1} との最大値をとって更新することで、正しく X_i を求めることができる。また、動的計画法の更新では、「木 i にはイルミネーションを飾り付けずに、以降の木を飾り付けるかどうかを決める」という遷移を許すことで、木 i にイルミネーションを飾り付けるとき、木 i から木 $X_i, X_i + 1, \dots$ への遷移をすべて試すかわりに、木 X_i への遷移のみを試せばよくなることがわかる。これにより、 $O(N+M)$ でこの問題を解くことができ、満点を獲得できる。

原文：<https://www2.ioi-jp.org/joi/2018/2019-yo/2019-yo-t5-review.html>

問題 6 座席 (Seats)

配点 100点 時間制限 5 sec メモリ制限 256 MB

解説

小課題 1

選手の人数の合計を $M = A_1 + \dots + A_N$ とおく。

単純な解法として、選手の並べ方を $M!$ 通りすべて試すという方法が考えられる。再帰関数あるいは C++ の `std::next_permutation` を用いるとよい。時間計算量は $O(M!)$ あるいは $O(M! \cdot M)$ などになる。

同じ国の選手を一旦区別せずに、国の番号の並べ方を考えることもできる。この場合、調べるのは $M! / (A_1! \dots A_N!)$ 通りとなり、最後に場合の数に $A_1! \dots A_N!$ をかければよい。

小課題 2

左から順番に並べる動的計画法による解法を考える。

「並べる選手の集合」と「右端の選手の国」の組に対して、条件を満たす並べ方が何通りあるか求めていく。同じ国の選手を一旦区別しない方針によって、選手の集合は各国の人数で表され、 $(A_1 + 1) \dots (A_N + 1)$ 通りとなり、 $O((A_1 + 1) \dots (A_N + 1) \times N^2)$ 時間で解くことができる。

この小課題では $A_i \leq 3$ であるから、各国の人数を $2N$ ビットの整数に格納することで、ビット演算を用いた効率的な実装が可能である。

小課題 3

国の番号が小さい順に並べ方を決めていく動的計画法による解法を考える。

途中の並べ方においては、条件を満たす並べ方でなくてもよい：例えば、国 3 まで並べたときに国 2 の選手どうしが隣り合っている、あとで間に国 4 の選手を挿入することで条件を満たすかもしれない。このように、「まだ条件を満たさない箇所がどのくらいあるか」を状態として動的計画法を行う。具体的には、国 $i - 1$ までを並べたとき、

- ・ 国番号 $i - 2$ 以下どうして、同じ国あるいは隣国で隣り合っている場所の個数
- ・ 国 $i - 1$ と国番号 $i - 3$ 以下が隣り合っている場所の個数（条件を満たさない場所ではないが、国 i の選手の挿入によって条件を満たさなくなる可能性があることに注意）
- ・ 国 $i - 1$ と国 $i - 2$ が隣り合っている場所の個数
- ・ 国 $i - 1$ どうして隣り合っている場所の個数

を保持する。ここに国 i の選手たちを加えるには、

- ・ 上記 4 種類のそれぞれについて、「国 i の連続」を何箇所に挿入するか
- ・ 上記 4 種類以外について、「国 i の連続」を何箇所に挿入するか

を決めると、状態を更新でき、場合の数も計算することができる（実際の式は複雑になるので、解答例を参照のこと）。この解法の時間計算量は $O(N (\max A_i)^9)$ となる。

上の解法では国ごとに並びに追加していったが、1 人ごとに追加していく考え方でも、同様に N と $\max A_i$ についての多項式時間で解くことができる（状態がやや複雑になるが、場合の数の計算は簡単になる）。

原文：<https://www2.ioi-jp.org/joi/2018/2019-yo/2019-yo-t6-review.html>

出典：情報オリンピック日本委員会「第18回日本情報オリンピック JOI 2018/2019 予選競技課題」。原資料は CC BY-SA 4.0 で提供されています。本PDFは各解説ページを問題順に再構成したものです。
<https://www2.ioi-jp.org/joi/2018/2019-yo/>